

Homework 6 Reference Solutions

- 7.2 a. False
 b. True
 c. True
 d. True
 e. False
 f. False

7.5 No, the formula is not satisfiable. Each assignment of Boolean values to x and y falsifies one of the four clauses.

7.7 NP is closed under union. For any two NP languages L_1 and L_2 , let M_1 and M_2 be the NTMs that decide them in polynomial time. We construct a NTM M' that decides $L_1 \cup L_2$ in polynomial time. We construct a TM M' that decides $L_1 \cup L_2$ in polynomial time:

$M' =$ "On input $\langle w \rangle$:

1. Run M_1 on w . If it accepts, *accept*.
2. Run M_2 on w . If it accepts, *accept*. Otherwise, *reject*."

In both steps 1 and 2, M' uses the non-determinism when the machines being run make nondeterministic steps. M' accepts w if and only if either M_1 or M_2 accepts w . Therefore, M' decides the union of L_1 and L_2 . Since both steps take polynomial time, the algorithm runs in polynomial time.

NP is closed under concatenation. For any two NP languages L_1 and L_2 , let M_1 and M_2 be the NTMs that decide them in polynomial time. We construct a NTM M' that decides $L_1 L_2$ in polynomial time.

$M' =$ "On input $\langle w \rangle$:

1. For each way to cut w into two substrings $w = w_1 w_2$.
2. Run M_1 on w_1 .
3. Run M_2 on w_2 . If both accept, *accept*; otherwise, continue with the next choice of w_1 and w_2 .
4. If w is not accepted after trying all the possible cuts, *reject*."

In both steps 2 and 3, M' uses its nondeterminism when the machines being run make nondeterministic steps. M' accepts w if and only if w can be expressed as $w_1 w_2$ such that M_1 accepts w_1 and M_2 accepts w_2 . Therefore, M' decides the concatenation of L_1 and L_2 . Step 2 runs in polynomial time and is repeated for at most $O(n)$ time where n is the length of the input, so the algorithm runs in polynomial time.

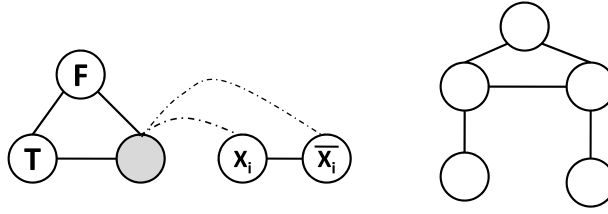


Figure 1: The palette, variable gadget, and OR-gadget.

7.21 $DOUBLE - SAT = \{ \langle \phi \rangle \mid \phi \text{ has at least two satisfying assignments} \}$.

On input ϕ , a nondeterministic polynomial time machine can guess two assignments and accept if both assignments satisfy ϕ . Thus $DOUBLE - SAT$ is in NP.

We show $SAT \leq_p DOUBLE - SAT$. The following TM F computes the polynomial time reduction f .

$F =$ "On input $\langle \phi \rangle$, a Boolean formula with variables $x_1, x_2, \dots, c_m$:

1. Let ϕ' be $\phi \wedge (x \vee \bar{x})$, where x is a new variable.
2. Output $\langle \phi' \rangle$."

If $\phi \in SAT$, then ϕ' has at least two satisfying assignments that can be obtained from the original satisfying assignments of ϕ by changing the value of x . If $\phi' \in DOUBLE - SAT$, then ϕ is also satisfiable, because x does not appear in ϕ . Therefore, $\phi \in SAT$ if and only if $f(\phi) \in DOUBLE - SAT$.

7.27 $3COLOR = \{ \langle G \rangle \mid \text{the nodes of } G \text{ can be colored with three colors such that no two nodes joined by an edge have the same color.} \}$

$3COLOR$ is in NP because a coloring can be verified in polynomial time. Next we show $3SAT \leq_p 3COLOR$. Let $\phi = c_1 \wedge c_2 \wedge \dots \wedge c_l$ be a 3CNF formula over variables $x_1, x_2, \dots, x_m$, where the c_i 's are the clauses. We build a graph G_ϕ containing $2m + 6l + 3$ nodes: 2 nodes for each variable; 6 nodes for each clause; and 3 extra nodes. We describe G_ϕ in terms of the subgraph gadgets given in Figure 1.

G_ϕ contains a variable gadget for each variable x_i , two OR-gadgets for each clause, and one palette gadget. The four bottom nodes of the OR-gadgets will be merged with other nodes in the graph, as follows. Label the nodes of the palette gadget T, F , and R . Label the nodes in each variable gadget $+$ and $-$, and connect each to the R node in the palette gadget. In each clause, connect the top of one of the OR-gadgets to the F node in the palette. Merge the bottom node of that OR-gadgets with the top node of the other OR-gadget. Merge the three remaining bottom nodes of the two OR-gadgets with corresponding nodes in the variable gadgets so that if a clause contains the literal x_i , one of its

bottom nodes is merged with the $+$ node of x_i whereas if the clause contains the literal $\bar{x}$, one of its bottom nodes is merged with the $-$ nodes of x_i . Figure 2 shows an small example for one clause and three variables.

To show the construction is correct, we first demonstrate that if ϕ is satisfiable, the graph is 3-colorable. The three colors are called T, F , and R . Color the palette with its labels. For each variable, color the $+$ node T and the $-$ node F if the variable is True in a satisfying assignment; otherwise, reverse the colors. Because each clause has one True literal in the assignment, we can color the nodes of the OR-gadgets of that clause so that the node connected to the F node in the palette is not colored F . Hence we have a proper 3-coloring.

If we start out with a 3-coloring, we can obtain a satisfying assignment by taking the colors assigned to the $+$ nodes of each variable. Observe that neither node of the variable gadget can be colored R , because all variable nodes are connected to the R node in the palette. Furthermore, if both bottom nodes of an OR-gadget are colored F , the top node must be colored F , and hence, each clause contains a true literal. Otherwise, the three bottom nodes that were merged with variable nodes would be colored F , and then both top nodes would be colored F , but one of the top nodes is connected to the F node in the palette.

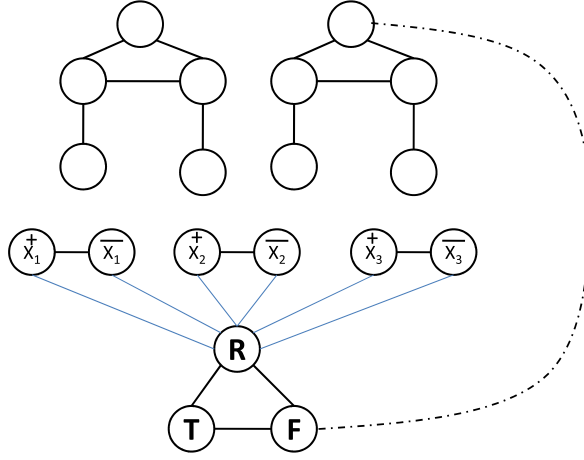
7.29 $SCHEDULE = \{ \langle F, S, h \rangle \mid F \text{ is a list of finals, } S \text{ is a list of subsets of finals each student is taking and finals can be scheduled into } h \text{ slots so that no student is taking two exams in the same slot.} \}$

$SCHEDULE$ is in NP because we can guess a schedule with h slots and verify in polynomial time that no student is taking two exams in the same slot. Next we reduce $3COLOR$ to $SCHEDULE$ in polynomial time. Give a graph (V, E) , transform it into $\langle F, S, h \rangle$ where $F = V$, $S = E$, $h = 3$.

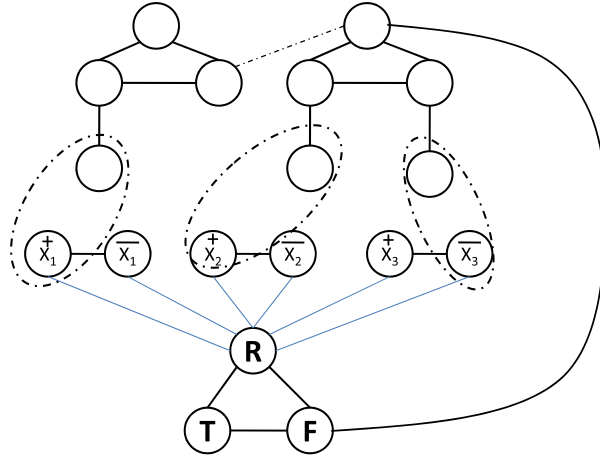
Given a 3-coloring for the original graph, we can get a schedule for $\langle V, E, 3 \rangle$ by assigning finals that correspond to nodes colored with color i to slot i for $i = 1, 2, 3$. Adjacent nodes are colored by different colors, so corresponding exams taken by the same student are assigned to different slots.

Given a schedule for $\langle V, E, 3 \rangle$, we can get a 3-coloring for the original graph by assigning colors according to the slots of corresponding nodes. Since exams taken by the same student are scheduled in different slots, adjacent nodes are assigned different colors.

$$X_1 \vee X_2 \vee X_3$$



$$X_1 \vee X_2 \vee X_3$$



$$X_1 \vee X_2 \vee X_3$$

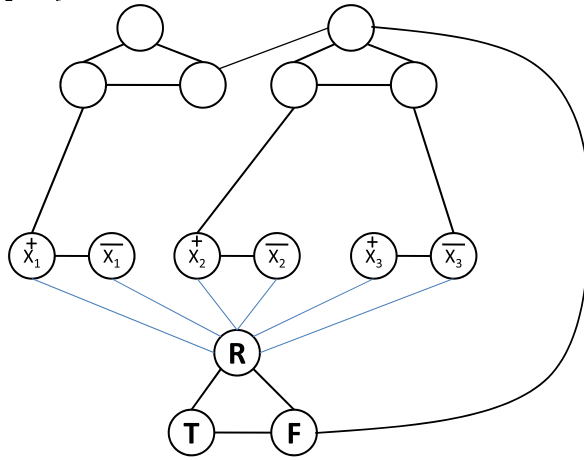


Figure 2: An example of the construction of G_ϕ .